OmniVoice: local voice synthesis, installation and a consented demo

Researched 2026-10-12.

What current project documentation confirms

The OmniVoice repository (https://github.com/k2-fsa/OmniVoice) documents PyPI installation, a Gradio demo and CLI inference. Install the PyTorch build appropriate to your device first using PyTorch's official selector (https://pytorch.org/get-started/locally/). Do not blindly copy a CUDA wheel line onto a different GPU or a CPU-only machine.

```
python -m pip install omnivoice
omnivoice-demo --ip 127.0.0.1 --port 8001
```

The loopback address intentionally keeps this prototype local. The project also supplies `omnivoice-infer`; consult its `--help` for your installed version. A first run needs network access and storage for weights. No installation or voice generation was executed here.

A practical first evaluation

1. Use a short clean recording you own or have explicit permission to use, plus an accurate reference transcript.
2. Generate a fixed set of ten short sentences, including names, numbers and punctuation. Keep the same settings between comparisons.
3. Compare intelligibility, pronunciation, speaker similarity, time to first output and total generation time on the actual device.
4. Test one target language at a time and record accent changes. A large supported-language count does not establish uniform quality.
5. Keep the original and synthetic samples labelled. Avoid a public upload as the default workflow.

Resource and access links

The repository links to its hosted Hugging Face demo, Python API, CLI reference and model. Follow those current links instead of a copied third-party mirror. Hosted demos have different privacy and queue behavior from local inference. The code license does not by itself establish rights to a person's voice or every downstream dataset.

Project idea

Create a local narration workbench with a text queue, explicit reference-audio selection, per-sample timing and a labelled export manifest. A useful acceptance check rejects missing permission metadata and handles model-download, device-memory and empty-audio failures. Compare against the existing Voicebox topic as a separate product; sharing a voice-cloning category does not make them duplicate implementations.

Reusable project prompt

```
Build Local narration workbench. Consented short samples, labelled synthetic outputs and device-specific performance measurements. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Verify device and PyTorch build; Run the local demo with permitted audio; Measure pronunciation and latency; Add labelled export manifest.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://www.instagram.com/reel/DdW1hsSMzVW/